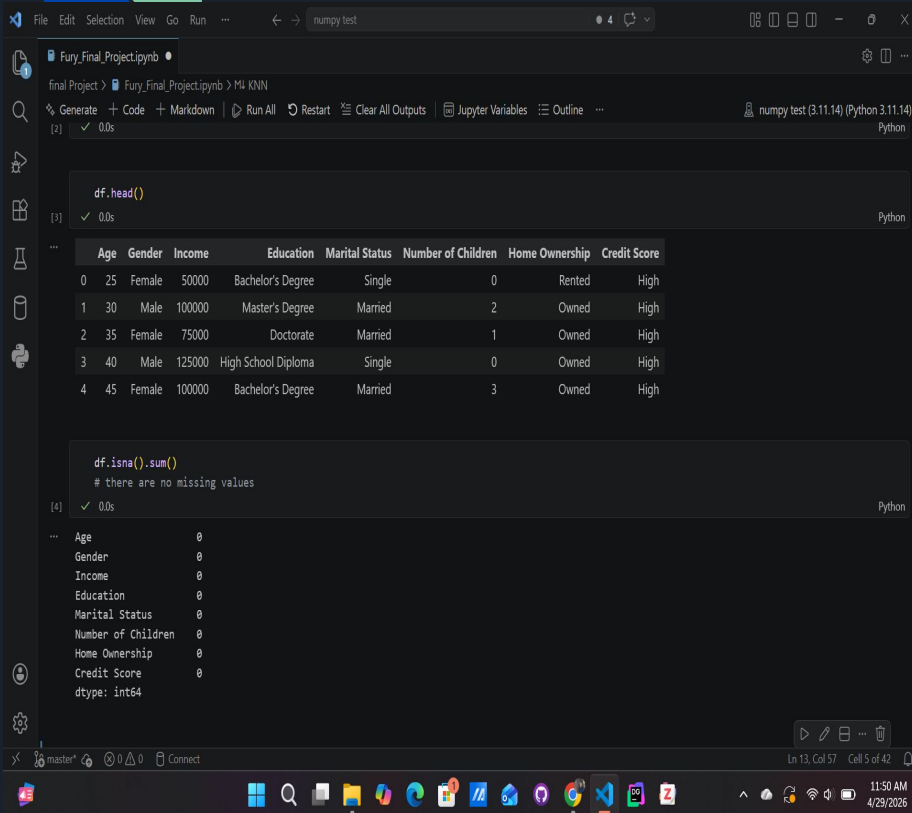


A decorative graphic on the left side of the slide. It consists of a blue parallelogram and a light green parallelogram, both tilted at an angle. The blue shape is in the foreground, and the green shape is partially behind it. They are set against a dark blue background with faint, lighter blue diagonal stripes.

Machine Learning Presentation

By Kaiden Fury

Code for reading the dataset and missing values



The left screenshot shows a Jupyter Notebook with the following content:

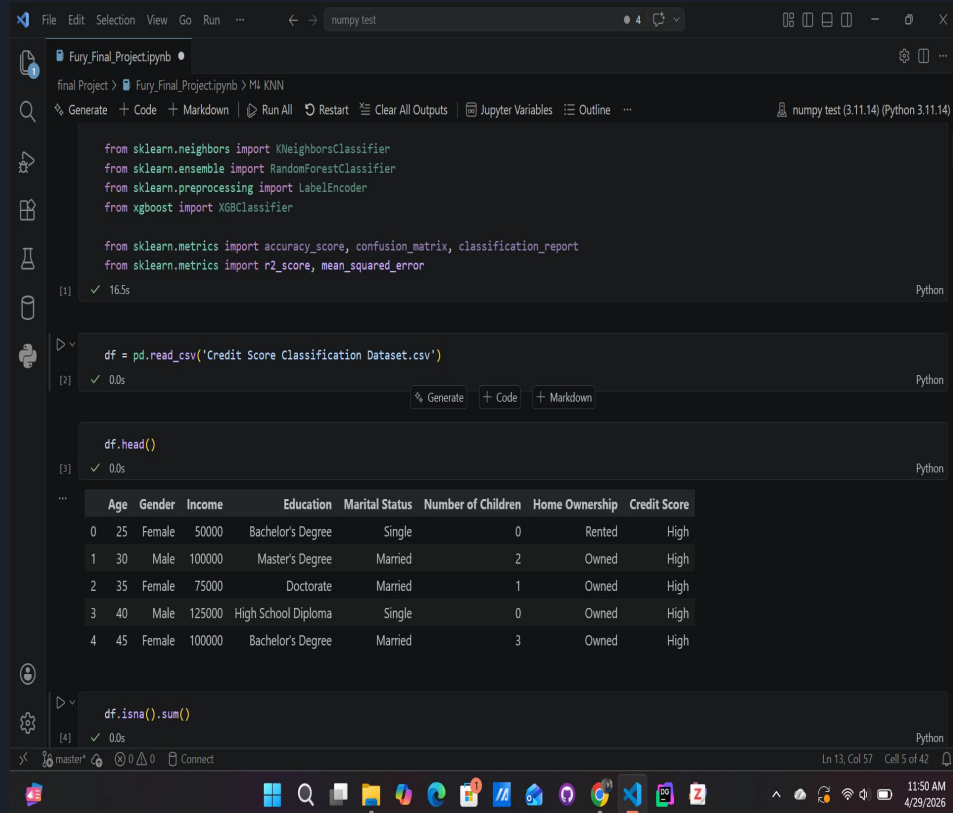
```
df.head()
```

	Age	Gender	Income	Education	Marital Status	Number of Children	Home Ownership	Credit Score
0	25	Female	50000	Bachelor's Degree	Single	0	Rented	High
1	30	Male	100000	Master's Degree	Married	2	Owned	High
2	35	Female	75000	Doctorate	Married	1	Owned	High
3	40	Male	125000	High School Diploma	Single	0	Owned	High
4	45	Female	100000	Bachelor's Degree	Married	3	Owned	High

```
df.isna().sum()
```

```
# there are no missing values
```

```
Age      0
Gender    0
Income    0
Education 0
Marital Status 0
Number of Children 0
Home Ownership 0
Credit Score 0
dtype: int64
```



The right screenshot shows a Jupyter Notebook with the following content:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import r2_score, mean_squared_error
```

```
df = pd.read_csv('Credit Score Classification Dataset.csv')
```

	Age	Gender	Income	Education	Marital Status	Number of Children	Home Ownership	Credit Score
0	25	Female	50000	Bachelor's Degree	Single	0	Rented	High
1	30	Male	100000	Master's Degree	Married	2	Owned	High
2	35	Female	75000	Doctorate	Married	1	Owned	High
3	40	Male	125000	High School Diploma	Single	0	Owned	High
4	45	Female	100000	Bachelor's Degree	Married	3	Owned	High

```
df.isna().sum()
```

Pictures of the KNN

```
df = df.dropna()

X = df.drop("Credit Score", axis=1)
y = df['Credit Score']

X = pd.get_dummies(X)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

Accuracy: 0.9393939393939394
precision recall f1-score support

Average 0.80 0.80 0.80 5
High 0.96 0.96 0.96 23
Low 1.00 1.00 1.00 5

accuracy 0.94 33
macro avg 0.92 0.92 0.92 33
weighted avg 0.94 0.94 0.94 33
```

Logistic Regression

Pictures of Logistic Regression

The image displays two side-by-side screenshots of a Jupyter Notebook interface, showing the implementation of Logistic Regression.

Left Screenshot:

- File:** Fury_Final_Project.ipynb
- Cell 4:** Logistic Regression
- Code:**

```
df = df.dropna()

X = df.drop("Credit Score", axis=1)
y = df["Credit Score"]

X = pd.get_dummies(X)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```
- Output:** [5] ✓ 0.0s, [6] ✓ 0.0s, [7] ✓ 0.0s

Right Screenshot:

- File:** Fury_Final_Project.ipynb
- Cell 4:** Logistic Regression
- Code:**

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report
print("Accuracy_LogisticRegression:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```
- Output:** [8] ✓ 0.0s, [9] ✓ 0.0s, [10] ✓ 0.0s
- Classification Report:**

```
Accuracy_LogisticRegression: 0.9393939393939394

precision    recall  f1-score   support

Average      0.80      0.80      0.80         5
High         0.96      0.96      0.96        23
Low          1.00      1.00      1.00         5

accuracy              0.94        33
macro avg           0.92      0.92      0.92        33
weighted avg        0.94      0.94      0.94        33
```

Pictures of Random Forest

The image shows a JupyterLab interface with two notebooks. The left notebook, titled 'Fury_Final_Project.ipynb', displays code for training a Random Forest classifier. The right notebook, also titled 'Fury_Final_Project.ipynb', shows the prediction and evaluation of the trained model.

Random forest

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)

rf.fit(X_train, y_train)
```

Parameters

```
y_pred = rf.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, classification_report

print("RandomForest_Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

RandomForest_Accuracy: 0.9090909090909091

	precision	recall	f1-score	support
Average	0.67	0.80	0.73	5
High	0.95	0.91	0.93	23
Low	1.00	1.00	1.00	5
accuracy			0.91	33
macro avg	0.87	0.90	0.89	33
weighted avg	0.92	0.91	0.91	33

XGBoost

XGBoost Pictures

The image displays two side-by-side screenshots of a Jupyter Notebook interface, illustrating the steps to implement XGBoost for classification.

Left Screenshot:

- Code Cell [62]:** Imports necessary libraries and prepares the data. It drops the 'Credit Score' column from the DataFrame and encodes categorical variables using `LabelEncoder`.

```
from sklearn.preprocessing import LabelEncoder
categorical_cols = ['Gender', 'Education', 'Marital Status', 'Home Ownership']
for col in categorical_cols:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])

# Encode target variable
le = LabelEncoder()
y = le.fit_transform(y)
```

- Code Cell [63]:** Imports `train_test_split` from `sklearn.model_selection` and splits the data into training and testing sets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Right Screenshot:

- Code Cell [64]:** Imports `XGBClassifier` from `xgboost` and creates a model with specific parameters.

```
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,
    random_state=42
)

model.fit(X_train, y_train)
```

- Code Cell [65]:** Uses the trained model to predict the results for the test set.

```
y_pred = model.predict(X_test)
```

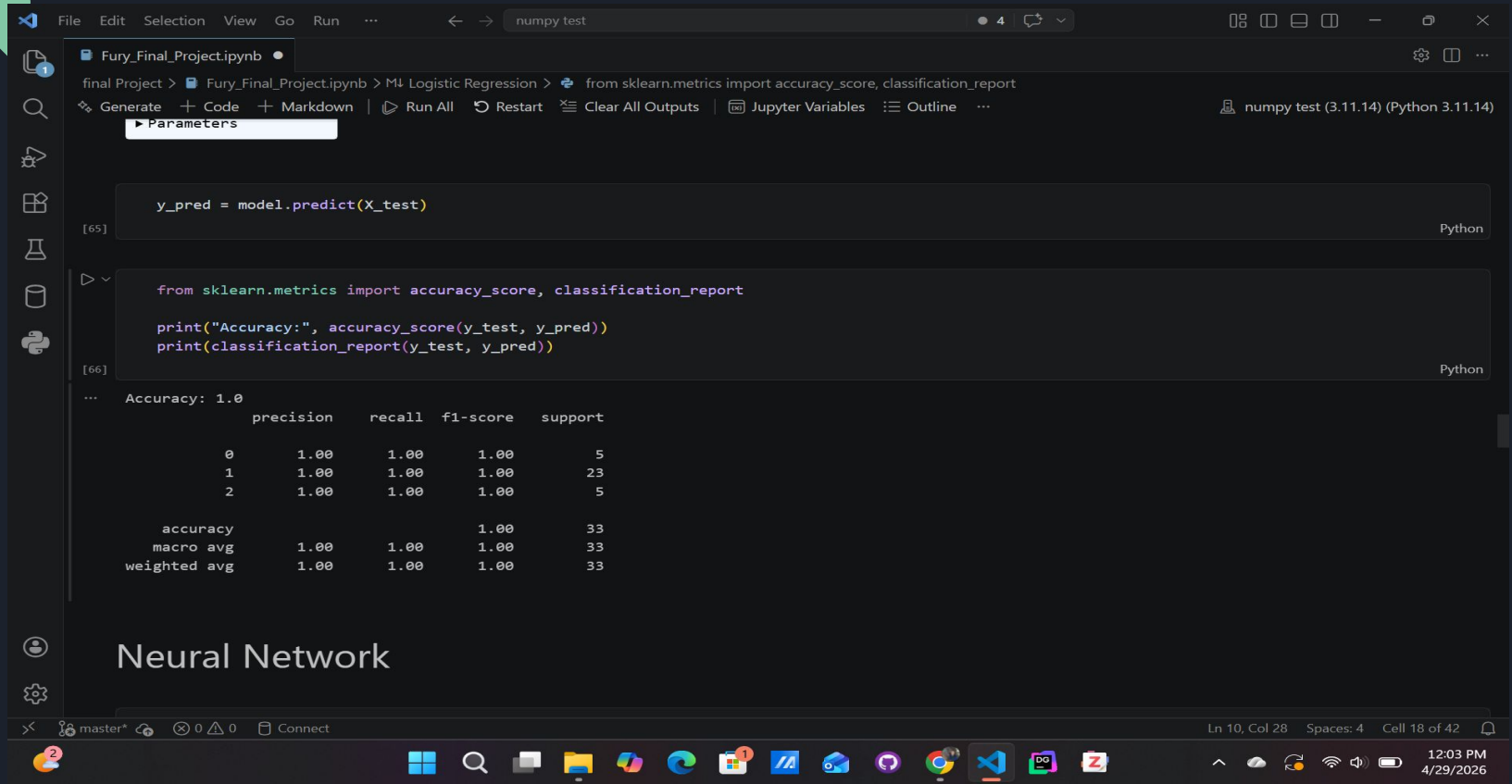
- Code Cell [66]:** Imports `accuracy_score` and `classification_report` from `sklearn.metrics` and prints the model's performance metrics.

```
from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

- Output:** The final output shows an **Accuracy: 1.0**, indicating perfect classification performance on the test set.

XGBoost Pictures Cont



The screenshot displays a Jupyter Notebook environment with the following components:

- File Explorer:** Shows the file `Fury_Final_Project.ipynb`.
- Code Editor:** Contains two code cells.
 - Cell [65]:** `y_pred = model.predict(X_test)`
 - Cell [66]:** Imports `accuracy_score` and `classification_report` from `sklearn.metrics`, and prints the accuracy and classification report for `y_test` and `y_pred`.
- Output:** The output of cell [66] shows the accuracy and classification report.

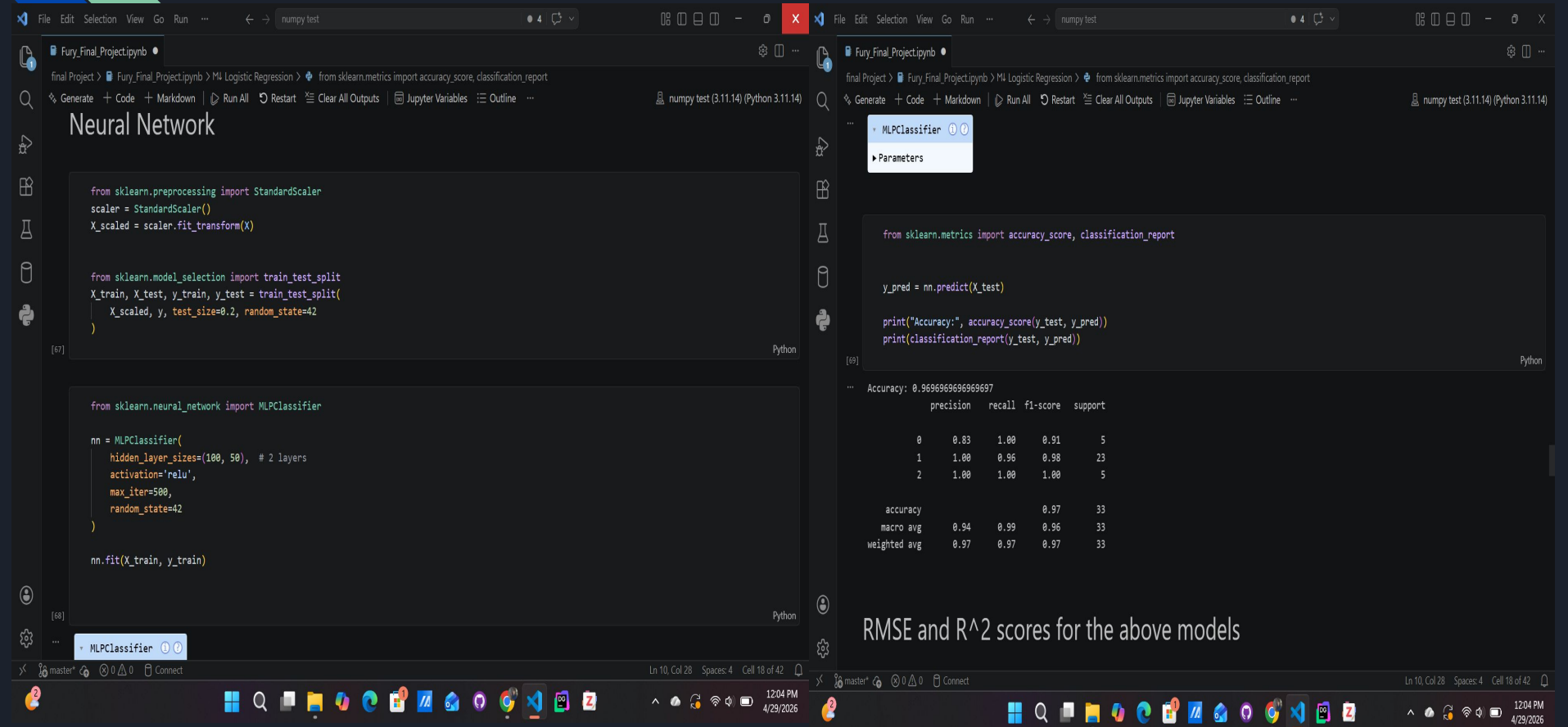
```
Accuracy: 1.00
precision    recall  f1-score   support

     0       1.00      1.00      1.00         5
     1       1.00      1.00      1.00        23
     2       1.00      1.00      1.00         5

 accuracy
macro avg       1.00      1.00      1.00        33
weighted avg     1.00      1.00      1.00        33
```
- Bottom Section:** A large text area with the heading `Neural Network`.

The bottom status bar indicates the current position is `Ln 10, Col 28` with `Spaces: 4` and `Cell 18 of 42`. The system clock shows `12:03 PM 4/29/2026`.

Neural Network Pictures



The image displays two side-by-side screenshots of a Jupyter Notebook interface, showing the training and evaluation of an MLPClassifier.

Left Screenshot:

- File:** Fury_Final_Project.ipynb
- Cell [57]:** Python code for preprocessing and model training.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

from sklearn.neural_network import MLPClassifier

nn = MLPClassifier(
    hidden_layer_sizes=(100, 50), # 2 layers
    activation='relu',
    max_iter=500,
    random_state=42
)

nn.fit(X_train, y_train)
```

Right Screenshot:

- File:** Fury_Final_Project.ipynb
- Cell [59]:** Python code for prediction and evaluation.

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = nn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output:

Accuracy: 0.9696969696969697

	precision	recall	f1-score	support
0	0.83	1.00	0.91	5
1	1.00	0.96	0.98	23
2	1.00	1.00	1.00	5
accuracy			0.97	33
macro avg	0.94	0.99	0.96	33
weighted avg	0.97	0.97	0.97	33

RMSE and R² scores for the above models

RMSE Score and R² score pictures

RMSE and R^2 scores for the above models

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.neural_network import MLPClassifier

knn = KNeighborsClassifier(n_neighbors=2)
log = LogisticRegression(max_iter=10000)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
nn = MLPClassifier(hidden_layer_sizes=(100,50), max_iter=200, random_state=42)

```

```
knn.fit(X_train, y_train)
log.fit(X_train, y_train)
rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)
nn.fit(X_train, y_train)
```

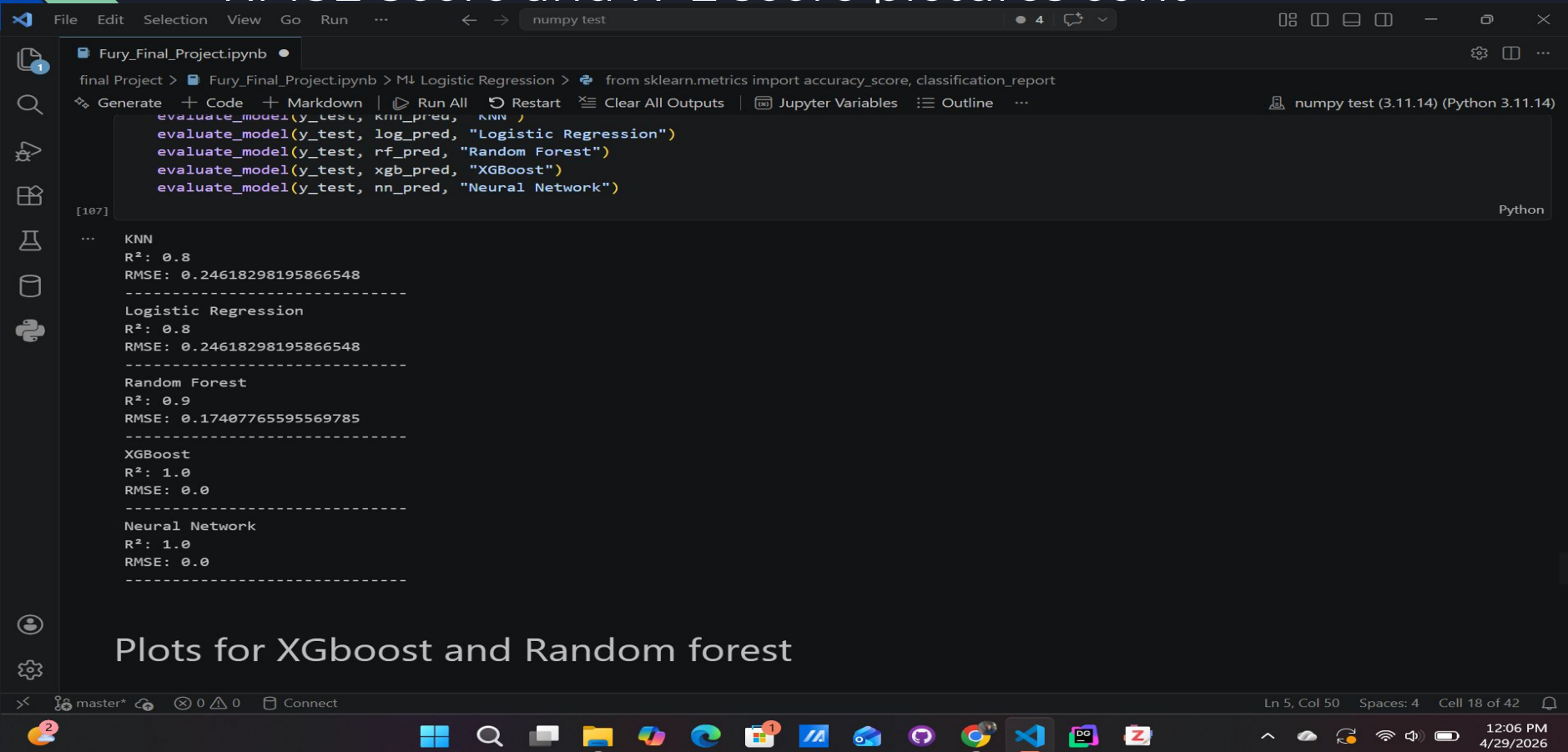
```
knn_pred = knn.predict(X_test)
log_pred = log.predict(X_test)
rf_pred = rf.predict(X_test)
xgb_pred = xgb.predict(X_test)
nn_pred = nn.predict(X_test)
```

```
def evaluate_model(y_test, y_pred, name):
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print(f"{name}")
    print("R²:", r2)
    print("RMSE:", rmse)
    print("-" * 30)
```

```
evaluate_model(y_test, knn_pred, "KNN")
evaluate_model(y_test, log_pred, "Logistic Regression")
evaluate_model(y_test, rf_pred, "Random Forest")
evaluate_model(y_test, xgb_pred, "XGBoost")
evaluate_model(y_test, nn_pred, "Neural Network")
```

RMSE Score and R^2 score pictures cont



The screenshot shows a Jupyter Notebook window with the following content:

```
final Project > Fury_Final_Project.ipynb > M4 Logistic Regression > from sklearn.metrics import accuracy_score, classification_report
Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ... numpy test (3.11.14) (Python 3.11.14)
```

```
evaluate_model(y_test, knn_pred, KNN)
evaluate_model(y_test, log_pred, "Logistic Regression")
evaluate_model(y_test, rf_pred, "Random Forest")
evaluate_model(y_test, xgb_pred, "XGBoost")
evaluate_model(y_test, nn_pred, "Neural Network")
```

[107] Python

```
...
KNN
R²: 0.8
RMSE: 0.24618298195866548
-----
Logistic Regression
R²: 0.8
RMSE: 0.24618298195866548
-----
Random Forest
R²: 0.9
RMSE: 0.17407765595569785
-----
XGBoost
R²: 1.0
RMSE: 0.0
-----
Neural Network
R²: 1.0
RMSE: 0.0
-----
```

Plots for XGboost and Random forest

Ln 5, Col 50 Spaces: 4 Cell 18 of 42

12:06 PM 4/29/2026

Plots for XGBoost and Random Forest Pics

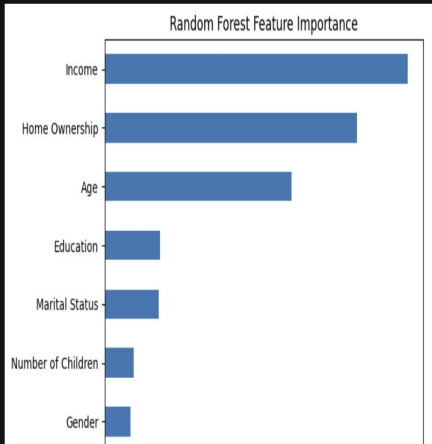
final Project > Fury_Final_ProjectIpynb > ML RMSE and R² scores for the above models > knn.fit(X_train, y_train)

```
import matplotlib.pyplot as plt
rf_importance = pd.Series(rf.feature_importances_, index=X.columns)
rf_importance = rf_importance.sort_values()

plt.figure()
rf_importance.plot(kind='barh')
plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.show()
```

[47] ✓ 0.2s

Python



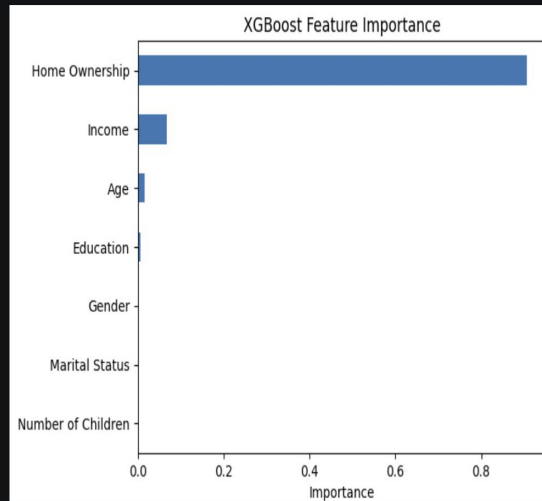
final Project > Fury_Final_ProjectIpynb > ML RMSE and R² scores for the above models > knn.fit(X_train, y_train)

```
xgb_importance = xgb_importance.sort_values()
```

```
plt.figure()
xgb_importance.plot(kind='barh')
plt.title("XGBoost Feature Importance")
plt.xlabel("Importance")
plt.show()
```

[48] ✓ 0.0s

Python



Ln 5, Col 50 Spaces: 4 Cell 36 of 42

Ln 8, Col 11 Spaces: 4 Cell 36 of 42



Conclusion and issues that was found

This project provided a valuable overview of all semester algorithms. Their accuracy scores were similar, with the neural network performing best. The challenge was determining the optimal RMSE and R-squared values for improved scoring for KNN and Logistic Regression. Other than that it was a good project and had a good time doing it.



Questions,
Comments,
Concerns?